

## Chapter 1: Introduction to Information Representation and Number Systems

### 1.1 Digital vs. Analog Systems

Digital and analog systems represent two different approaches to processing information. Both are essential in various areas, but digital systems have gained prominence due to their precision, reliability, and versatility.

**Digital Systems:** These systems process discrete data that is typically represented in binary form, consisting of two distinct values (0 and 1). They operate based on Boolean logic and are less prone to noise and distortion compared to analog systems. Computers, microcontrollers, and communication systems rely heavily on digital processing for efficiency and error correction.

**Analog Systems:** Analog systems, on the other hand, deal with continuous data. The signals in these systems vary continuously and can take an infinite number of values within a given range. Examples of analog systems include traditional audio signals and radio transmissions. Analog systems are particularly useful for applications where a high degree of precision is not crucial, and the system must respond to real-world phenomena, such as sound or light.

While digital systems are deterministic and can be easily manipulated for complex computations, analog systems excel in representing real-world variations with fewer quantization errors. Understanding the distinction between the two is crucial in digital logic design as we focus on discrete logic operations.

### 1.2 Number Systems

Digital systems primarily work with **binary numbers** (base 2), but other number systems like **decimal** (base 10), **octal** (base 8), and **hexadecimal** (base 16) are frequently used in digital design for convenience and compactness.

#### 1.2.1 Binary System (Base-2)

The **binary system** uses two digits: 0 and 1. Each digit in a binary number is referred to as a **bit**. In the binary system:

The rightmost bit is the **least significant bit** (LSB).

The leftmost bit is the **most significant bit** (MSB).

**Example:** The binary number  $1101_2$  is equal to:

$$(1 \times 2^3) + (1 \times 2^2) + (0 \times 2^1) + (1 \times 2^0) = 8 + 4 + 1 = 13$$
$$10(1 \times 2^3) + (1 \times 2^2) + (0 \times 2^1) + (1 \times 2^0) = 8 + 4 + 1 = 13_{10}$$

#### 1.2.2 Decimal System (Base-10)

The **decimal system** is the number system most familiar to humans, consisting of ten digits (0-9). It is a positional system, meaning that the value of a digit depends on its position relative to the decimal point.

**Example:** The decimal number  $253_{10}$  is equal to:

$$(2 \times 10^2) + (5 \times 10^1) + (3 \times 10^0) = 200 + 50 + 3 = 253_{10} \\ (2 \times 10^2) + (5 \times 10^1) + (3 \times 10^0) = 200 + 50 + 3 = 253_{10}$$

### 1.2.3 Octal System (Base-8)

The **octal system** is a positional number system with a base of 8. It uses the digits 0 to 7. The octal system is often used in digital systems to simplify the representation of binary numbers.

**Example:** The octal number  $17_8$  is equal to:

$$(1 \times 8^1) + (7 \times 8^0) = 8 + 7 = 15_{10} \\ (1 \times 8^1) + (7 \times 8^0) = 8 + 7 = 15_{10}$$

### 1.2.4 Hexadecimal System (Base-16)

The **hexadecimal system** uses 16 digits: 0-9 and A-F, where A represents 10, B represents 11, and so on up to F, which represents 15. This system is frequently used in digital systems because it provides a more compact representation of binary numbers.

**Example:** The hexadecimal number  $2F_{16}$  is equal to:

$$(2 \times 16^1) + (15 \times 16^0) = 32 + 15 = 47_{10} \\ (2 \times 16^1) + (15 \times 16^0) = 32 + 15 = 47_{10}$$

### 1.2.5 Number System Conversion

Understanding how to convert between different number systems is essential for digital logic design. Here are the general conversion techniques:

**Binary to Decimal:** Multiply each bit by the power of 2 corresponding to its position and sum the results.

**Decimal to Binary:** Divide the decimal number by 2, record the remainder, and repeat until the quotient is 0. The binary equivalent is the remainders read in reverse order.

**Binary to Octal:** Group the binary number in sets of three bits from right to left and convert each group to its octal equivalent.

**Binary to Hexadecimal:** Group the binary number in sets of four bits from right to left and convert each group to its hexadecimal equivalent.

## 1.3 Binary Arithmetic

Binary arithmetic follows the same principles as decimal arithmetic, but it is based on the binary number system.

### 1.3.1 Binary Addition

Binary addition is performed using the following rules:

$$0 + 0 = 0$$

$$0 + 1 = 1$$

$$1 + 0 = 1$$

$$1 + 1 = 10 \text{ (carry 1)}$$

**Example:**

$$11012+10112=1100021101_2 + 1011_2 = 11000_2$$

### 1.3.2 Binary Subtraction

Binary subtraction follows these rules:

$$0 - 0 = 0$$

$$1 - 0 = 1$$

$$1 - 1 = 0$$

0 - 1 requires a borrow.

**Example:**

$$10102-01102=010021010_2 - 0110_2 = 0100_2$$

### 1.3.3 Binary Multiplication

Binary multiplication follows these rules:

$$0 \times 0 = 0$$

$$0 \times 1 = 0$$

$$1 \times 0 = 0$$

$$1 \times 1 = 1$$

**Example:**

$$1012 \times 112 = 11112101_2 \times 11_2 = 1111_2$$

### 1.3.4 Binary Division

Binary division is similar to long division in decimal. The quotient is obtained by repeatedly dividing and subtracting the divisor from the dividend.

## 1.4 Signed Number Representation

In digital systems, **signed numbers** are used to represent both positive and negative values. This requires special methods for encoding negative numbers.

#### 1.4.1 1's Complement Representation

The **1's complement** of a binary number is obtained by inverting all the bits (0s become 1s and 1s become 0s).

This method can be used to represent negative numbers.

##### Example:

The 1's complement of 0110 is 1001.

#### 1.4.2 2's Complement Representation

The **2's complement** is the most common way to represent negative numbers in digital systems. It is calculated by taking the 1's complement of a number and adding 1 to the result.

##### Example:

The 2's complement of 0110 is 1010 (invert the bits, then add 1).

#### 1.4.3 Sign-Magnitude Representation

In **sign-magnitude representation**, the leftmost bit is used as the sign bit (0 for positive, 1 for negative), and the remaining bits represent the magnitude.

##### Example:

+6 in sign-magnitude is 00000110.

-6 in sign-magnitude is 10000110.

#### Flashbook for Chapter 1:

| Mnemonic | Meaning                                          |
|----------|--------------------------------------------------|
| BIN      | Binary Number System (Base-2)                    |
| DEC      | Decimal Number System (Base-10)                  |
| OCT      | Octal Number System (Base-8)                     |
| HEX      | Hexadecimal Number System (Base-16)              |
| 1's Comp | Invert all bits for negative numbers             |
| 2's Comp | Invert and add 1 for negative numbers            |
| SM       | Sign-Magnitude representation for signed numbers |



## Chapter 2: Boolean Algebra and Switching Theory

### 2.1 Introduction to Boolean Algebra

**Boolean algebra** is a mathematical framework for analyzing and designing digital circuits. It is based on binary variables that take the values **0** (false) or **1** (true). The primary operations of Boolean algebra are **AND**, **OR**, and **NOT**, which correspond to logical operations in digital systems.

Boolean algebra allows us to simplify complex expressions and design circuits with fewer components, thereby optimizing hardware.

#### 2.1.1 Basic Boolean Operations

There are three fundamental operations in Boolean algebra:

**AND (  $\wedge$  )**: The result is true only if both operands are true.

$A \cdot B = \text{True if } A = 1 \text{ and } B = 1$

**Truth Table:**

| A | B | $A \wedge B$ |
|---|---|--------------|
| 0 | 0 | 0            |
| 0 | 1 | 0            |
| 1 | 0 | 0            |
| 1 | 1 | 1            |

**OR (  $\vee$  )**: The result is true if at least one operand is true.

$A + B = \text{True if } A = 1 \text{ or } B = 1$

**Truth Table:**

| A | B | $A \vee B$ |
|---|---|------------|
| 0 | 0 | 0          |
| 0 | 1 | 1          |
| 1 | 0 | 1          |
| 1 | 1 | 1          |

**NOT (  $\neg$  )**: The result is the opposite of the operand. If the operand is true, it becomes false, and if the operand is false, it becomes true.

$\neg A = \text{True if } A = 0 \text{ and } \neg A = 0 \text{ if } A = 1$

Truth Table:

| A | $\neg A$ |
|---|----------|
| 0 | 1        |
| 1 | 0        |

|   |   |
|---|---|
| 0 | 1 |
| 1 | 0 |

### 2.1.2 Derived Boolean Operations

**NAND (Not AND):** The result is true unless both inputs are true.

$$A \text{ NAND } B = \neg(A \cdot B) \quad \text{NAND } B = \neg(A \cdot B)$$

**Truth Table:**

| A | B | A NAND B |
|---|---|----------|
| 0 | 0 | 1        |
| 0 | 1 | 1        |
| 1 | 0 | 1        |
| 1 | 1 | 0        |

**NOR (Not OR):** The result is true only when both inputs are false.

$$A \text{ NOR } B = \neg(A + B) \quad \text{ NOR } B = \neg(A + B)$$

**Truth Table:**

| A | B | A NOR B |
|---|---|---------|
| 0 | 0 | 1       |
| 0 | 1 | 0       |
| 1 | 0 | 0       |
| 1 | 1 | 0       |

**XOR (Exclusive OR):** The result is true only when exactly one operand is true.

$$A \oplus B = (A \cdot \neg B) + (\neg A \cdot B) \quad A \oplus B = (A \cdot \neg B) + (\neg A \cdot B)$$

Truth Table:

| A | B | A XOR B |
|---|---|---------|
| 0 | 0 | 0       |
| 0 | 1 | 1       |
| 1 | 0 | 1       |
| 1 | 1 | 0       |

**XNOR (Exclusive NOR):** The result is true when both operands are the same.

$$A \odot B = \neg(A \oplus B) \quad A \odot B = \neg(A \oplus B)$$

Truth Table:

| A | B | A XNOR B |
|---|---|----------|
| 0 | 0 | 1        |
| 0 | 1 | 0        |
| 1 | 0 | 0        |
| 1 | 1 | 1        |

## 2.2 Laws of Boolean Algebra



Boolean algebra has several important laws that help simplify complex expressions, making it easier to design and optimize digital circuits.

### 2.2.1 Identity Law

$$A \cdot 1 = A \quad A \vee 0 = A$$

$$A + 0 = A \quad A \cdot 1 = A$$

### 2.2.2 Null Law

$$A \cdot 0 = 0 \quad A \vee 1 = 1$$

$$A + 1 = 1 \quad A \cdot 0 = 0$$

### 2.2.3 Complement Law

$$A \cdot \neg A = 0 \quad A \vee \neg A = 1$$

$$A + \neg A = 1 \quad A \cdot \neg A = 0$$

### 2.2.4 Idempotent Law

$$A \cdot A = A \quad A \vee A = A$$

$$A + A = A \quad A \cdot A = A$$

### 2.2.5 Domination Law

$$A \cdot 0 = 0 \quad A \vee 1 = 1$$

$$A + 1 = 1 \quad A \cdot 0 = 0$$

### 2.2.6 Absorption Law

$$A \cdot (A + B) = A \quad A + (A \cdot B) = A$$

$$A + (A \cdot B) = A \quad A \cdot (A + B) = A$$

### 2.2.7 Distributive Law

$$A \cdot (B + C) = (A \cdot B) + (A \cdot C) \quad A + (B \cdot C) = (A + B) \cdot (A + C)$$

$$A + (B \cdot C) = (A + B) \cdot (A + C) \quad A \cdot (B + C) = (A \cdot B) + (A \cdot C)$$

### 2.2.8 De Morgan's Theorems

$$\neg(A \cdot B) = \neg A + \neg B \quad \neg(A + B) = \neg A \cdot \neg B$$

$$\neg(A + B) = \neg A \cdot \neg B \quad \neg(A \cdot B) = \neg A + \neg B$$

## 2.3 Simplification of Boolean Expressions

Simplification of Boolean expressions is crucial in digital circuit design because simpler expressions lead to circuits with fewer components, making them more efficient. Several methods are used for simplification, including **Boolean algebraic laws**, **Karnaugh maps (K-maps)**, and **Quine–McCluskey method**.

### 2.3.1 Karnaugh Maps (K-maps)

Karnaugh maps are a graphical method to simplify Boolean expressions. A K-map is a two-dimensional grid where each cell represents a unique combination of input variables. By grouping adjacent cells with the same value (either 1 or 0), you can simplify the Boolean expression.

#### Example:

Given the Boolean function  $f(A,B,C)=A \cdot B + \neg A \cdot C$ , you would represent this in a K-map, grouping the ones and applying the simplification rules.

### 2.3.2 Quine–McCluskey Method

This method is an algorithmic approach to minimizing Boolean functions. It is typically used when K-maps become too complex to handle manually, especially with a large number of variables.

## 2.4 Switching Functions

A **switching function** is a Boolean function used to represent a logical decision in digital circuits. Switching functions are the foundation of digital circuit design and are defined by a truth table or Boolean expression.

For example, a function could represent whether a light should be on (1) or off (0) based on the state of various switches.

## 2.5 Canonical Forms

Canonical forms are standardized representations of Boolean functions. There are two types:

**Sum of Products (SOP):** A Boolean expression where terms are OR-ed together, and each term is an AND of literals.

$$f(A,B,C)=A \cdot B + \neg A \cdot C \quad f(A, B, C) = A \cdot B + \neg A \cdot C$$

**Product of Sums (POS):** A Boolean expression where terms are AND-ed together, and each term is an OR of literals.

$$f(A,B,C)=(A+B) \cdot (\neg A+C) \quad f(A, B, C) = (A + B) \cdot (\neg A + C)$$

Flashbook for Chapter 2:

| Mnemonic | Meaning |
|----------|---------|
|----------|---------|

|              |                                                  |
|--------------|--------------------------------------------------|
| AND          | The result is true if both inputs are true       |
| OR           | The result is true if at least one input is true |
| NOT          | The result is the inverse of the operand         |
| NAND         | NOT AND (true unless both inputs are true)       |
| NOR          | NOT OR (true only when both inputs are false)    |
| XOR          | True when exactly one operand is true            |
| XNOR         | True when both operands are the same             |
| Laws of Bool | Identity, Complement, Distributive, etc.         |
| K-map        | Graphical method to simplify Boolean expressions |
| SOP/POS      | Sum of Products / Product of Sums                |

## Chapter 3: Minimization and Gate Characteristics

### 3.1 Minimization of Boolean Functions

Minimizing Boolean functions is a critical step in designing efficient digital circuits. The goal is to reduce the complexity of a Boolean expression, which leads to a simpler circuit with fewer gates. Two main techniques for minimization are **Boolean algebraic simplification** and **Karnaugh maps (K-maps)**.

#### 3.1.1 Boolean Algebraic Simplification

Boolean algebraic simplification involves applying the laws of Boolean algebra to reduce the number of terms in a Boolean expression. By systematically using rules such as the identity law, absorption law, and distributive law, we can minimize the number of gates required in a circuit.

**Example:** Simplify the Boolean expression:

$$f(A,B,C)=A \cdot B + A \cdot \neg C + B \cdot C \quad f(A, B, C) = A \cdot B + A \cdot \neg C + B \cdot C$$

First, we factor out common terms:

$$f(A,B,C)=A \cdot (B + \neg C) + B \cdot C \quad f(A, B, C) = A \cdot (B + \neg C) + B \cdot C$$

This is a simplified version of the original function with fewer terms, reducing the complexity of the circuit.

#### 3.1.2 Karnaugh Maps (K-maps)

Karnaugh maps provide a visual approach to simplifying Boolean functions. The K-map is a two-dimensional grid that represents all possible combinations of input variables. By grouping adjacent cells containing 1s (for SOP) or 0s (for POS), we can find the simplest Boolean expression.

#### Steps for Simplifying with K-maps:

Plot the function in a K-map.

Group adjacent cells that contain 1s (or 0s, depending on the type of simplification).

Derive the simplified expression from the groups.

Minimize the number of terms and literals.

#### 3.1.3 Quine–McCluskey Method

The **Quine–McCluskey** method is an algorithmic technique for minimizing Boolean expressions, particularly when dealing with functions with a large number of variables. This method is systematic and works by comparing terms to find common factors and progressively eliminating redundant terms.

Example:

For a function with four variables, we would:

List all possible minterms.

Combine minterms that differ by only one variable.

Continue combining terms until no further combinations are possible.

Identify the essential prime implicants and write the simplified Boolean expression.

### 3.2 Gate Characteristics

The physical characteristics of digital gates are crucial for understanding how they behave in real circuits. The most important characteristics of gates include **fan-in**, **fan-out**, **propagation delay**, **timing diagrams**, and **tri-state drivers**.

#### 3.2.1 Fan-in and Fan-out

**Fan-in** refers to the number of input signals a gate can handle. For instance, a 2-input AND gate has a fan-in of 2. As the number of inputs increases, the gate becomes more complex, and the likelihood of errors increases.

**Fan-out** refers to the number of outputs that a gate can drive. A gate can drive a limited number of other gates without degrading signal quality. If the fan-out is too high, the output voltage levels may not be sufficient to drive subsequent gates.

#### 3.2.2 Propagation Delay

Propagation delay is the time it takes for a signal to propagate through a gate. It is an important factor in determining the speed of digital circuits. In synchronous circuits, propagation delay affects the clock period and the overall system timing.

**Propagation delay (tpd)** is typically measured from the point where the input changes to when the output changes.

Minimizing propagation delay is crucial for high-speed systems. Modern digital circuits aim for very low propagation delays to achieve faster processing speeds.

#### 3.2.3 Timing Diagrams

A **timing diagram** is a graphical representation of the behavior of signals over time. It is used to illustrate how inputs and outputs of a circuit change in relation to a clock signal.

##### Key Elements of a Timing Diagram:

**Clock signal:** A periodic signal that synchronizes the circuit.

**Input signals:** Signals that change based on the input data.

**Output signals:** Signals that reflect the results of the logic operation.

Timing diagrams help in visualizing how different gates and flip-flops behave within a clocked system.

Example:

Consider a simple circuit with an AND gate and a flip-flop. The timing diagram would show how the input signals change, how the output of the AND gate responds to the inputs, and how the flip-flop stores the output on the rising edge of the clock signal.

#### 3.2.4 Tri-state Drivers

A **tri-state driver** is a special type of output driver that can be in one of three states: **high**, **low**, or **high-impedance** (Z). The high-impedance state is similar to an open circuit, meaning the output does not drive a signal.

Tri-state drivers are commonly used in **bus systems**, where multiple devices share a common set of wires. When a device is not actively transmitting data, it enters the high-impedance state to avoid interfering with other devices on the bus.

### 3.3 Practical Considerations

In practical digital circuit design, understanding gate characteristics and how to minimize Boolean expressions is key to creating efficient systems. Designers often balance between circuit complexity, power consumption, speed, and reliability.

#### 3.3.1 Gate Implementation

Gates are implemented using physical components like transistors in **CMOS (Complementary Metal-Oxide-Semiconductor)** technology. The characteristics of these gates, including their speed, fan-in, and fan-out, influence the overall design of a system.

#### 3.3.2 Power Consumption

Minimizing the number of gates and optimizing the layout can significantly reduce power consumption. Power optimization is critical for portable and battery-operated devices, where energy efficiency is a priority.

Flashbook for Chapter 3:

| <b>Mnemonic</b> | <b>Meaning</b>                                    |
|-----------------|---------------------------------------------------|
| SOP             | Sum of Products                                   |
| POS             | Product of Sums                                   |
| K-map           | Karnaugh Map, a graphical simplification tool     |
| Tpd             | Propagation Delay                                 |
| Fan-in          | Number of inputs to a gate                        |
| Fan-out         | Number of outputs a gate can drive                |
| Tri-state       | High, low, and high-impedance output states       |
| Quine-McCluskey | Algorithm for minimizing Boolean functions        |
| Timing Diagrams | Visual representation of signal changes over time |

## Chapter 4: Combinational and Sequential Circuit Design

### 4.1 Combinational Circuits

#### 4.1.1 Characteristics

- Output depends only on current input
- No memory or feedback

#### 4.1.2 Key Components

- **MUX:** Multiple inputs, one output, selected by control lines
- **DEMUX:** One input routed to multiple outputs
- **Decoders:**  $n$  inputs  $\rightarrow 2^n$  outputs
- **Encoders:**  $2^n$  inputs  $\rightarrow n$  outputs
- **Comparators:** Outputs  $A > B$ ,  $A = B$ ,  $A < B$
- **Adders:**
  - Half Adder: Sum, Carry
  - Full Adder: Includes carry-in
- **Subtractors:** Implemented using adders and 2's complement

### 4.2 Sequential Circuits

#### 4.2.1 Flip-Flops

- **SR:** Set-Reset
- **JK:** Toggle when  $J = K = 1$
- **D:** Data flip-flop (most used)
- **T:** Toggles output

#### 4.2.2 Clocking

- Controls flip-flop timing
- Edge-triggered vs level-triggered

#### 4.2.3 Timing Diagrams

- Show transitions of inputs/outputs



- Used for designing reliable systems

### 4.3 Registers and Counters

#### 4.3.1 Registers

- Storage unit of flip-flops
- Shift and parallel-load types

#### 4.3.2 Counters

- **Asynchronous:** Ripple counters, slower
- **Synchronous:** Faster, simultaneous triggering

### 4.4 Memory and Programmable Devices

#### 4.4.1 RAM

- Volatile, read-write memory

#### 4.4.2 ROM

- Non-volatile, read-only memory

#### 4.4.3 PLAs/PLDs

- Fixed AND array, programmable OR (or vice versa)

#### 4.4.4 FPGAs

- Large programmable devices
- Implement complex digital systems

### Flashbook for Chapter 4:

| Mnemonic  | Meaning                   |
|-----------|---------------------------|
| MUX/DEMUX | Multiplexer/Demultiplexer |
| DEC/ENC   | Decoder/Encoder           |

|           |                               |
|-----------|-------------------------------|
| REG/CNT   | Register/Counter              |
| FF        | Flip-Flop                     |
| SR/JK/D/T | Flip-Flop Types               |
| PLA/PLD   | Programmable Logic Devices    |
| FPGA      | Field-Programmable Gate Array |

## Chapter 5: Digital System Design Principles and Applications

### 5.1 System Design Methodology

- **Specification:** Define what the system should do
- **Design:** Create the logic circuit using gates
- **Simulation:** Verify correctness using tools like ModelSim
- **Implementation:** Hardware realization using FPGAs or PLDs

### 5.2 Hierarchical Design

- Break large systems into subsystems
- Promotes modularity and reuse

### 5.3 State Machines

- Finite state machines (FSM) define system behavior with a finite number of states
- Two types:
  - **Moore:** Output depends only on state
  - **Mealy:** Output depends on state and input

### 5.4 Applications of Digital Logic

- **Computers:** CPU design, memory units, buses
- **Embedded Systems:** Automotive, medical devices, IoT
- **Consumer Electronics:** TVs, mobile phones, gaming devices
- **Control Systems:** Robotics, automation

### 5.5 Design Tools and Languages

- **HDLs:** Hardware Description Languages like VHDL and Verilog
- **Simulation Tools:** ModelSim, Quartus, Vivado

### 5.6 Testing and Verification

- Essential to ensure reliability
- **Test benches:** Used in simulation
- **In-circuit testing:** Physical tests on hardware

**Flashbook for Chapter 5:**

| Mnemonic    | Meaning                       |
|-------------|-------------------------------|
| FSM         | Finite State Machine          |
| HDL         | Hardware Description Language |
| VHDL        | VHSIC HDL                     |
| Moore/Mealy | FSM Types                     |
| Sim/Impl    | Simulation/Implementation     |
| PLD/FPGA    | Programmable Devices          |